

Software Engineering Practical Lab Manual

Complete 10 Practical Experiments with Java, JUnit 5, TDD, Git SCM, COCOMO & Cyclomatic Testing

Author: Shaswat Raj (BIT Mesra CSE) Academic Scheme: NEP 2024–25 (5th Sem) Credits: 1.5 Practical Credits

Experiment 1: IEEE 830-1998 Software Requirements Specification (SRS)

Objective: Elicit, analyze, and document complete Functional and Non-Functional Requirements (FURPS+) following the IEEE 830 standard for a distributed Hospital Management System (HMS).

IEEE 830 Standard SRS Section 3 Specification Template

- 3.1 External Interface Requirements:** User Interfaces (React/Tailwind), Hardware Interfaces (Barcode Scanner, Biometric Sensor), Software Interfaces (PostgreSQL 15, Redis Cache), Communications Interfaces (HTTPS RESTful APIs, TLS 1.3).
- 3.2 Functional Requirements:**
 - FR-01: The system shall authenticate doctors via JWT bearer tokens within ≤ 200 ms.
 - FR-02: The system shall lock patient prescription records during concurrent modifications using Redis distributed mutexes.
- 3.3 Non-Functional Requirements:** Availability $\geq 99.99\%$, Throughput ≥ 5000 req/sec, Sub-second query response latency.

Experiment 2: Intermediate COCOMO & COCOMO II Cost Estimation in Python

Objective: Implement Boehm's Intermediate COCOMO software cost estimation model in Python, adjusting nominal effort by multiplying 15 Cost Drivers (Effort Multipliers EM_i).

```
class COCOMOEstimator:
    # Model parameters for Organic, Semi-Detached, and Embedded
    MODELS = {
        'Organic':      {'a': 2.4, 'b': 1.05, 'c': 2.5, 'd': 0.38},
        'Semi-Detached': {'a': 3.0, 'b': 1.12, 'c': 2.5, 'd': 0.35},
        'Embedded':     {'a': 3.6, 'b': 1.20, 'c': 2.5, 'd': 0.32}
    }

    def estimate(self, kloc, mode='Semi-Detached', cost_drivers=None):
        if cost_drivers is None: cost_drivers = [1.0] * 15
        params = self.MODELS[mode]
        eaf = 1.0
        for em in cost_drivers: eaf *= em # Effort Adjustment Factor

        effort_pm = params['a'] * (kloc ** params['b']) * eaf
        tdev_months = params['c'] * (effort_pm ** params['d'])
        staff_size = effort_pm / tdev_months
        productivity = (kloc * 1000) / effort_pm

        return {
            'Mode': mode,
            'KLOC': kloc,
            'Effort (Person-Months)': round(effort_pm, 2),
            'Development Time (Months)': round(tdev_months, 2),
            'Average Staff Required': round(staff_size, 1),
            'Productivity (LOC/PM)': round(productivity, 1)
        }

calc = COCOMOEstimator()
# Project of 45 KLOC with RELY=1.15, CPLX=1.15, TIME=1.11, EXP=0.91
drivers = [1.15, 1.0, 1.15, 1.0, 1.11, 1.0, 1.0, 0.91, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
res = calc.estimate(45, mode='Semi-Detached', cost_drivers=drivers)
for k, v in res.items(): print(f"{k:26}: {v}")
```

Experiment 3: Albrecht Function Point Analysis (FPA) in Python

Objective: Calculate Unadjusted Function Points (UFP), Value Adjustment Factor (VAF), and Adjusted Function Points (AFP) based on 14 General System Characteristics (GSC).

```
def calculate_function_points(counts, complexity_weights, gsc_scores):
    # Counts: [EI, EO, EQ, ILF, EIF]
    # Complexity: Matrix of weights [Simple, Average, Complex]
    ufp = sum(c * w for c, w in zip(counts, complexity_weights))
    tdi = sum(gsc_scores) # Total Degree of Influence (0 to 70)
    vaf = 0.65 + 0.01 * tdi # Value Adjustment Factor
    afp = ufp * vaf
    return {'UFP': ufp, 'TDI': tdi, 'VAF': round(vaf, 4), 'Adjusted FP': round(afp, 2)}

# 10 EI (Avg=4), 6 EO (Avg=5), 4 EQ (Avg=4), 3 ILF (Avg=10), 2 EIF (Avg=7)
ufp_counts = [10, 6, 4, 3, 2]
weights = [4, 5, 4, 10, 7]
gsc = [3, 4, 2, 4, 3, 2, 4, 3, 2, 3, 4, 3, 2, 3] # 14 factors scored 0-5

fp_result = calculate_function_points(ufp_counts, weights, gsc)
print("Function Point Analysis Result:", fp_result)
```

Experiment 4: CPM / PERT Project Scheduling & Critical Path Analysis

Objective: Construct an Activity-on-Arrow (AOA) scheduling network, compute Earliest Start (ES), Earliest Finish (EF), Latest Start (LS), Latest Finish (LF), Total Float (Slack), and find the Critical Path.

```
class Activity:
    def __init__(self, name, duration, preds):
        self.name = name
        self.duration = duration
        self.preds = preds
        self.succs = []
        self.es = self.ef = self.ls = self.lf = self.slack = 0

def critical_path_method(activities):
    act_dict = {a.name: a for a in activities}
    for a in activities:
        for p in a.preds: act_dict[p].succs.append(a.name)

    # 1. Forward Pass (Earliest Times)
    for a in activities:
        if not a.preds:
            a.es = 0
            a.ef = a.duration
        else:
            a.es = max(act_dict[p].ef for p in a.preds)
            a.ef = a.es + a.duration

    project_duration = max(a.ef for a in activities)

    # 2. Backward Pass (Latest Times)
    for a in reversed(activities):
        if not a.succs:
            a.lf = project_duration
            a.ls = a.lf - a.duration
        else:
            a.lf = min(act_dict[s].ls for s in a.succs)
            a.ls = a.lf - a.duration
        a.slack = a.ls - a.es

    critical_path = [a.name for a in activities if a.slack == 0]
    return project_duration, critical_path

tasks = [
    Activity('A', 4, []),
    Activity('B', 6, ['A']),
    Activity('C', 3, ['A']),
    Activity('D', 7, ['B']),
    Activity('E', 5, ['B', 'C']),
    Activity('F', 2, ['D', 'E'])
]
dur, cp = critical_path_method(tasks)
print(f"Total Project Duration: {dur} Weeks | Critical Path: {' → '.join(cp)}")
```

Experiment 5: Automated Unit Testing with JUnit 5 & Mockito in Java

Objective: Implement Test-Driven Development (TDD) using JUnit 5 assertions, parameterized tests, and Mockito dynamic service mocks in Java.

```
// BankAccountTest.java (JUnit 5 & Mockito Suite)
import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class BankAccountTest {
    private BankAccount account;
    private NotificationService mockNotifier;

    @BeforeEach
    void setUp() {
        mockNotifier = mock(NotificationService.class);
        account = new BankAccount("ACC-101", 1000.0, mockNotifier);
    }

    @Test
    @DisplayName("Deposit increases balance and triggers SMS notification")
    void testDepositSuccess() {
        account.deposit(500.0);
        assertEquals(1500.0, account.getBalance(), 0.001);
        verify(mockNotifier, times(1)).sendAlert(eq("ACC-101"), contains("500.0"));
    }

    @ParameterizedTest
    @ValueSource(doubles = {-10.0, 0.0, -500.0})
    @DisplayName("Invalid deposit amounts throw IllegalArgumentException")
    void testInvalidDeposit(double amount) {
        assertThrows(IllegalArgumentException.class, () -> account.deposit(amount));
    }

    @Test
    @DisplayName("Withdrawal exceeding balance throws InsufficientFundsException")
    void testOverdraw() {
        assertThrows(InsufficientFundsException.class, () -> account.withdraw(2000.0));
        assertEquals(1000.0, account.getBalance(), 0.001); // Balance unchanged
    }
}
```

Experiment 6: McCabe Cyclomatic Complexity & Basis Path Test Suite in C

Objective: Construct Control Flow Graphs (CFG), compute McCabe Cyclomatic Complexity $V(G) = E - N + 2P$, and derive the linearly independent basis path set.

```
#include <stdio.h>

// Function: Find Maximum of Three Numbers
int max3(int a, int b, int c) {
    int max;
    if (a > b) {           // Decision Node 1
        if (a > c)         // Decision Node 2
            max = a;
        else
            max = c;
    } else {
        if (b > c)         // Decision Node 3
            max = b;
        else
            max = c;
    }
    return max;
}

/*
CFG Analysis:
- Decision Nodes (P) = 3
- McCabe Complexity V(G) = P + 1 = 3 + 1 = 4 Basis Paths!

Basis Paths:
Path 1: (a > b: True) -> (a > c: True) -> max = a (Test: a=5, b=3, c=2)
Path 2: (a > b: True) -> (a > c: False) -> max = c (Test: a=5, b=3, c=8)
Path 3: (a > b: False) -> (b > c: True) -> max = b (Test: a=2, b=7, c=4)
Path 4: (a > b: False) -> (b > c: False) -> max = c (Test: a=2, b=7, c=9)
*/
```

Experiment 7: Black-Box Boundary Value Analysis (BVA) in Python

Objective: Generate $4n + 1$ Single-Fault Boundary Value Test Cases and $6n + 1$ Robustness Test Cases for a numeric date input system ($1 \leq \text{Day} \leq 31$, $1 \leq \text{Month} \leq 12$, $1900 \leq \text{Year} \leq 2050$).

```
def generate_bva_test_cases(min_v, max_v, var_name):
    # Single fault values: min, min+1, nom, max-1, max
    nom = (min_v + max_v) // 2
    return [
        (min_v, f"{var_name} = Min ({min_v})"),
        (min_v + 1, f"{var_name} = Min+1 ({min_v+1})"),
        (nom, f"{var_name} = Nominal ({nom})"),
        (max_v - 1, f"{var_name} = Max-1 ({max_v-1})"),
        (max_v, f"{var_name} = Max ({max_v})")
    ]

# Robustness adds: min-1 and max+1
def generate_robust_bva(min_v, max_v):
    nom = (min_v + max_v) // 2
    return [min_v - 1, min_v, min_v + 1, nom, max_v - 1, max_v, max_v + 1]

days = generate_robust_bva(1, 31)
print("Boundary Value Analysis Robustness Vectors for Day (1..31):", days)
```

Experiment 8: Git SCM Branching, Merging & GitHub Actions CI/CD

Objective: Implement industrial Git Software Configuration Management (SCM): feature branching, semantic version tagging, merge conflict resolution, and automated continuous integration pipeline.

```
# 1. Initialize repository and create release baseline branch
git init
git checkout -b main
git commit --allow-empty -m "chore: initial baseline commit"

# 2. Develop on feature branch
git checkout -b feature/jwt-auth
# ... modify auth code ...
git commit -am "feat: implement JWT token authentication"

# 3. Rebasing onto main and tag production release
git checkout main
git merge --no-ff feature/jwt-auth -m "merge: integrate feature/jwt-auth"
git tag -a v1.0.0 -m "Production Release Baseline 1.0.0"
```

GitHub Actions CI Pipeline Configuration (`.github/workflows/ci.yml`)

```
name: Java CI with Maven & JUnit 5
on: [push, pull_request]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Build with Maven
        run: mvn clean test
      - name: Publish Test Results
        uses: EnricoMi/publish-unit-test-result-action@v2
        if: always()
        with:
          files: target/surefire-reports/*.xml
```

Experiment 9: UML 2.5 Architectural Modeling: Class & Use Case Diagrams

Objective: Design complete UML 2.5 structural specifications in PlantUML: Class hierarchies with multiplicity, abstract classes, composition (◆), aggregation (◇), and dependency injection.

```
@startuml
skinparam classAttributeIconSize 0

abstract class User {
    - id: String
    - name: String
    - email: String
    + authenticate(token: String): boolean
}

class Patient {
    - medicalRecordNumber: String
    - dateOfBirth: Date
    + viewPrescriptions(): List
    + bookAppointment(doc: Doctor, time: Date): Appointment
}

class Doctor {
    - specialization: String
    - licenseNumber: String
    + writePrescription(p: Patient, meds: List): Prescription
}

class Prescription {
    - issueDate: Date
    - validUntil: Date
    + getDetails(): String
}

User <|-- Patient
User <|-- Doctor
Doctor "1" --> "*" Prescription : creates >
Patient "1" *-- "*" Prescription : owns (Composition)
@enduml
```

Experiment 10: UML 2.5 Dynamic Interaction: Sequence & Activity Diagrams

Objective: Construct dynamic behavioral interaction diagrams showing message flows across User, Gateway, Auth Service, and Database with `alt` / `opt` execution frames.

```
@startuml
autonumber
actor Patient as P
boundary "API Gateway" as GW
control "Auth Service" as Auth
database "PostgreSQL DB" as DB

P -> GW: POST /login {email, password}
activate GW
GW -> Auth: validateCredentials(email, hash)
activate Auth
Auth -> DB: SELECT user, hash FROM users WHERE email=?
activate DB
DB --> Auth: user_record
deactivate DB

alt Valid Password
    Auth --> GW: 200 OK + JWT Signed Token
    GW --> P: 200 OK {token, expiresIn: 3600}
else Invalid Password
    Auth --> GW: 401 Unauthorized
    GW --> P: 401 Unauthorized {error: "Bad Credentials"}
end
deactivate Auth
deactivate GW
@enduml
```

Experiment 11: Risk Management (RMMM Plan) Matrix in Python

Objective: Build an automated Risk Exposure calculator ($RE = P \times C$) and prioritize software project risks into High, Medium, and Low mitigation priority queues.

```
class RiskItem:
    def __init__(self, name, probability, impact_cost, mitigation_strategy):
        self.name = name
        self.prob = probability # 0.0 to 1.0
        self.impact = impact_cost # INR / USD
        self.exposure = probability * impact_cost
        self.mitigation = mitigation_strategy

risks = [
    RiskItem("Staff Turnover (Lead Architect Leaves)", 0.30, 500000, "Cross-training & comprehensive architecture doc"),
    RiskItem("Scope Creep (20% Feature Addition)", 0.60, 300000, "Formal CCB approval & agile sprint re-estimation"),
    RiskItem("Cloud Database Outage", 0.05, 1000000, "Multi-region active-active database replication"),
    RiskItem("Third-Party API Deprecation", 0.25, 200000, "Adapter design pattern & fallback mock services")
]

# Sort by Risk Exposure descending
sorted_risks = sorted(risks, key=lambda r: r.exposure, reverse=True)
print(f"{'Risk Description':40} | {'Prob':4} | {'Impact':8} | {'Exposure':8} | Priority")
print("-" * 80)
for r in sorted_risks:
    prio = "HIGH" if r.exposure >= 100000 else "MED"
    print(f"{r.name:40} | {r.prob:.2f} | {r.impact:8d} | {r.exposure:8.0f} | [{prio}]")
```

Experiment 12: Halstead Software Science Metrics Engine in Python

Objective: Parse tokenized source code in Python to compute Halstead Program Vocabulary (η), Length (N), Volume (V), Difficulty (D), and Development Effort (E).

```
import math

def calculate_halstead(n1, n2, N1, N2):
    # n1: Unique operators, n2: Unique operands
    # N1: Total operators, N2: Total operands
    vocab = n1 + n2
    length = N1 + N2
    volume = length * math.log2(vocab)
    difficulty = (n1 / 2.0) * (N2 / float(n2))
    effort = difficulty * volume
    time_sec = effort / 18.0
    bugs = volume / 3000.0
    return {
        'Program Vocabulary (eta)': vocab,
        'Program Length (N)': length,
        'Volume (V bits)': round(volume, 2),
        'Difficulty (D)': round(difficulty, 2),
        'Effort (E)': round(effort, 2),
        'Est. Time (Seconds)': round(time_sec, 2),
        'Delivered Bugs (B)': round(bugs, 4)
    }

metrics = calculate_halstead(n1=14, n2=8, N1=35, N2=20)
for k, v in metrics.items(): print(f"{k:26}: {v}")
```

Experiment 13: Software Reliability & Operational Availability Modeling

Objective: Implement exponential failure distribution models in Python to derive Reliability $R(t) = e^{-\lambda t}$, Failure Density $f(t)$, Hazard Rate $Z(t)$, and Operational Availability $A = \frac{MTTF}{MTTF+MTTR}$.

```
import math

def calculate_reliability(mttf_hours, mttr_hours, operational_time_t):
    failure_rate_lambda = 1.0 / mttf_hours
    reliability = math.exp(-failure_rate_lambda * operational_time_t)
    mtbf = mttf_hours + mttr_hours
    availability = mttf_hours / mtbf
    return {
        'Failure Rate (failures/hr)': round(failure_rate_lambda, 6),
        f'Reliability at t={operational_time_t}h': f"{reliability * 100:.2f}%",
        'MTBF (Hours)': mtbf,
        'Operational Availability (A)': f"{availability * 100:.4f}%"
    }

metrics = calculate_reliability(mttf_hours=990.0, mttr_hours=10.0, operational_time_t=100.0)
for k, v in metrics.items(): print(f"{k:32}: {v}")
```

Experiment 14: Defect Removal Efficiency (DRE) & Maintainability Index

Objective: Calculate Defect Removal Efficiency ($DRE = \frac{E}{E+D} \times 100\%$) and the Coleman-Oman Maintainability Index ($MI = 171 - 5.2\ln(V) - 0.23V(G) - 16.2\ln(LOC)$).

```
import math

def calculate_quality_metrics(errors_pre_delivery, defects_post_delivery, volume_v, cyclomatic_vg, loc):
    dre = (errors_pre_delivery / (errors_pre_delivery + defects_post_delivery)) * 100.0
    mi = 171 - 5.2 * math.log(volume_v) - 0.23 * cyclomatic_vg - 16.2 * math.log(loc)
    return {
        'Defect Removal Efficiency (DRE)': f"{dre:.2f}%",
        'Maintainability Index (MI)': round(mi, 2),
        'Maintainability Rating': 'HIGH' if mi ≥ 85 else ('MODERATE' if mi ≥ 65 else 'DIFFICULT')
    }

q_res = calculate_quality_metrics(errors_pre_delivery=140, defects_post_delivery=10,
                                volume_v=1250.0, cyclomatic_vg=6, loc=450)
for k, v in q_res.items(): print(f"{k:32}: {v}")
```

Experiment 15: Clean Architecture & SOLID Principle Verification in Java

Objective: Implement a Dependency Inversion Principle (DIP) and Interface Segregation Principle (ISP) decoupled architecture in Java to isolate business domain entities from external framework drivers.

```
// OrderPaymentService.java (Clean SOLID Architecture)
public interface PaymentGateway {
    PaymentReceipt charge(String customerId, double amount);
}

public class StripePaymentGateway implements PaymentGateway {
    @Override
    public PaymentReceipt charge(String customerId, double amount) {
        // Stripe API integration logic
        return new PaymentReceipt("TXN-STRIPE-9941", amount, "SUCCESS");
    }
}

public class OrderCheckoutUseCase {
    private final PaymentGateway gateway; // Dependency Inversion (DIP)

    public OrderCheckoutUseCase(PaymentGateway gateway) {
        this.gateway = gateway;
    }

    public OrderConfirmation execute(Order order) {
        PaymentReceipt receipt = gateway.charge(order.getCustomerId(), order.getTotalAmount());
        order.markPaid(receipt.getTransactionId());
        return new OrderConfirmation(order.getId(), receipt.getStatus());
    }
}
```

Experiment 16: UML 2.5 State Machine Diagram Modeling for ATM Controller

Objective: Model dynamic state transitions, guard conditions `[condition]`, entry/exit actions, and composite nested states for an ATM Banking System in PlantUML.

```
@startuml
[*] → Idle

state Idle {
    Idle : entry / displayWelcomeScreen()
}

Idle → CardInserted : insertCard(cardData)
state CardInserted {
    CardInserted : entry / readChip()
}

CardInserted → PinPrompt : cardValid [pinAttempts < 3]
CardInserted → EjectCard : cardDamaged / displayError()

state PinPrompt {
    PinPrompt : entry / promptPIN()
}

PinPrompt → Authenticated : enterPin(pin) [verifyPin = true]
PinPrompt → PinPrompt : enterPin(pin) [verifyPin = false && attempts < 3] / incrementAttempts()
PinPrompt → ConfiscateCard : enterPin(pin) [attempts ≥ 3] / alertSecurity()

state Authenticated {
    [*] → SelectTransaction
    SelectTransaction → DispenseCash : selectWithdraw(amt) [balance ≥ amt]
    DispenseCash → PrintReceipt : cashDispensed
}

Authenticated → EjectCard : logout / endSession()
EjectCard → Idle : cardRemoved
ConfiscateCard → Idle : reset()
@enduml
```

Experiment 17: Decision Table Testing for E-Commerce Dynamic Discount Engine

Objective: Construct a complete 2^N condition Decision Table and implement the dynamic rules engine in Python to test complex boolean business rules.

```
def calculate_discount(is_prime_member, cart_total, is_festive_sale, has_coupon):
    # Rule 1: Prime + Cart > 2000 + Festive → 30% discount
    if is_prime_member and cart_total > 2000 and is_festive_sale:
        return 0.30
    # Rule 2: Prime + Coupon → 20% discount
    if is_prime_member and has_coupon:
        return 0.20
    # Rule 3: Cart > 3000 + Festive → 15% discount
    if cart_total > 3000 and is_festive_sale:
        return 0.15
    # Rule 4: Coupon only → 10% discount
    if has_coupon:
        return 0.10
    return 0.0 # Default no discount

# Decision Table Test Suite Execution
test_cases = [
    (True, 2500, True, False, 0.30, "Rule 1: Super Prime Festive"),
    (True, 1200, False, True, 0.20, "Rule 2: Prime Coupon"),
    (False, 3500, True, False, 0.15, "Rule 3: Non-Prime Bulk Festive"),
    (False, 500, False, True, 0.10, "Rule 4: Standard Coupon"),
    (False, 800, False, False, 0.00, "Rule 5: Default Base")
]

print(f"{'Test Description':32} | {'Expected':8} | {'Actual':8} | Status")
print("-" * 65)
for prime, total, fest, coup, expected, desc in test_cases:
    actual = calculate_discount(prime, total, fest, coup)
    status = "PASS" if abs(actual - expected) < 1e-4 else "FAIL"
    print(f"{desc:32} | {expected*100:6.1f}% | {actual*100:6.1f}% | [{status}]")
```


Experiment 18: Gang-of-Four (GoF) Design Patterns Implementation in Java

Objective: Implement the thread-safe Double-Checked Locking Singleton pattern and the loose-coupling Observer publish-subscribe pattern in Java.

```
// ThreadSafeSingleton.java
public class DatabaseConnectionPool {
    private static volatile DatabaseConnectionPool instance;

    private DatabaseConnectionPool() {
        // Private constructor prevents direct instantiation
    }

    public static DatabaseConnectionPool getInstance() {
        if (instance == null) {
            synchronized (DatabaseConnectionPool.class) {
                if (instance == null) {
                    instance = new DatabaseConnectionPool();
                }
            }
        }
        return instance;
    }
}

// ObserverPattern.java
import java.util.ArrayList;
import java.util.List;

interface OrderEventListener {
    void onOrderStatusChanged(String orderId, String newStatus);
}

class OrderNotificationHub {
    private final List listeners = new ArrayList<>();

    public void subscribe(OrderEventListener listener) { listeners.add(listener); }
    public void notifyAll(String orderId, String status) {
        for (OrderEventListener l : listeners) l.onOrderStatusChanged(orderId, status);
    }
}
```

Comprehensive Viva-Voce Question Bank & Model Answers

Q1. What is the difference between Verification and Validation?

- **Verification ("Are we building the product right?"):** Static evaluation of process artifacts (reviews, inspections, static code analysis) to ensure compliance with phase specifications without executing code.
- **Validation ("Are we building the right product?"):** Dynamic evaluation of the running software against customer user requirements and operational expectations.

Q2. How is McCabe's Cyclomatic Complexity calculated and what does it indicate?

Calculated as $V(G) = E - N + 2P = P_{\text{decision_nodes}} + 1$. It defines the exact upper bound for the number of linearly independent execution paths required to achieve 100% branch coverage!

Q3. What is the difference between High Cohesion and Low Coupling?

High Cohesion means elements within a single module strongly relate and focus on a single well-defined task (Functional Cohesion is ideal).
Low Coupling means modules have minimal, explicit dependencies through clean data interfaces (Data Coupling is ideal, Content Coupling must be avoided).

Q4. What is a Baseline in Software Configuration Management (SCM)?

A **Baseline** is a formally reviewed and agreed-upon version of a configuration item (SRS, Architecture, Source Code) that serves as the basis for further development, and can only be modified through the formal Change Control Board (CCB) approval process!

Q5. Differentiate between Aggregation and Composition in UML Class Diagrams.

- **Aggregation (Hollow Diamond ◇):** "Has-a" relationship with weak ownership and independent lifecycle (e.g., Department ◇ → Professor; if department closes, professors still exist).
- **Composition (Filled Diamond ◆):** "Part-of" relationship with strict ownership and coincident lifecycle (e.g., House ◆ → Room; if house is destroyed, rooms are destroyed).

Q6. Explain the 5 Maturity Levels of SEI CMMI.

1. **Initial (Ad-hoc):** Chaotic, unpredictable budget/schedule.
2. **Managed (Repeatable):** Basic project management baselines.
3. **Defined (Standardized):** Organization-wide engineering processes.
4. **Quantitatively Managed (Measured):** Statistical process control & metrics.
5. **Optimizing (Continuous Improvement):** Defect prevention & agile process innovation.

Q7. What is Modified Condition/Decision Coverage (MC/DC) and why is it used in FAA DO-178C?

MC/DC requires testing each condition within a decision outcome to show that it can independently affect the decision outcome while all other conditions are held fixed. It achieves near-exhaustive safety verification with only $N + 1$ test cases rather than 2^N exponential condition permutations!